Exp: 1

In Linux, **system calls** provide an interface between a user program and the **kernel**. The functions you listed are commonly used in C/C++ system programming.

System call	Purpose
fork()	Creates a new child process by duplicating the calling process. Returns 0 in the child and the child's PID in the parent.
exec()	Replaces the current process with a new program. In C, this is a family such as execl(), execv(), execvp(), etc.
getpid()	Returns the **process ID (PID)** of the calling process.
pipe()	Creates a one-way communication channel between processes. It provides two file descriptors: one for reading and one for writing.
exit()	Terminates the calling process and returns an exit status to its parent. Strictly, exit() is a C library function; _exit() is the underlying system-level termination function.
open()	Opens or creates a file and returns a **file descriptor**.
close()	Closes an open file descriptor and releases its associated resources.
stat()	Retrieves information about a file, such as its size, permissions, owner, and modification time.
uname()	Retrieves information about the operating system/kernel and machine.

Important syntax examples

```
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/utsname.h>

pid_t pid = fork();                 // create process
pid_t mypid = getpid();             // get process ID

int fd[2];
pipe(fd);                           // create pipe

int f = open("file.txt", O_RDONLY); // open file
close(f);                           // close file

struct stat s;
stat("file.txt", &s);               // get file information

struct utsname u;
uname(&u);                          // get system information
```

A typical process-creation sequence is:

```
Parent process
     |
   fork()
    / \
Parent Child
         |
       exec()
         |
    New program
```

```
        |
      exit()
```

The key idea is that fork() **creates a process,** exec() **loads a program into a process,** pipe() **enables communication,** open()**/**close() **manage files, and** stat()**/**uname() **retrieve system information.**

Exp: 2

Process-Related Commands in Linux

Linux provides several commands and system calls for **monitoring, controlling, scheduling, and managing processes**.

Command / Function	Purpose	Example
top	Displays running processes dynamically, including CPU and memory usage.	top
ps	Displays information about currently running processes.	ps aux
kill	Sends a signal to a process, commonly used to terminate it.	kill 1234
wait	Makes a shell/script wait for a background process to finish.	wait $!
sleep	Suspends execution for a specified amount of time.	sleep 5
nice	Starts a process with a specified CPU scheduling priority.	nice -n 10 ./program
renice	Changes the priority of an already-running process.	renice 10 -p 1234
bg	Resumes a stopped job in the background.	bg %1
fg	Brings a background/stopped job to the foreground.	fg %1

1. top

top provides a **real-time view of running processes**.

```
top
```

It displays information such as:

- Process ID (PID)
- CPU usage
- Memory usage
- Process owner
- Process state

Press q to quit.

2. ps

ps displays a snapshot of processes.

```
ps
```

Some commonly used forms:

```
ps -ef
ps aux
```

For example, ps -ef provides detailed information about processes running on the system.

3. kill

kill sends a signal to a process.

```
kill 1234
```

By default, it sends SIGTERM, requesting the process to terminate.

To force termination:

```
kill -9 1234
```

Here, -9 sends SIGKILL.

4. wait

wait is primarily a **shell built-in**. It waits for a background process to complete.

```
./program &
wait
echo "Program finished"
```

You can also wait for a specific process:

```
wait 1234
```

5. sleep

sleep pauses execution for a specified period.

```
sleep 5
```

This pauses for **5 seconds**.

Other examples:

```
sleep 2m
sleep 1h
```

6. nice

nice starts a program with a particular **niceness value**, which influences CPU scheduling priority.

```
nice -n 10 ./program
```

A higher niceness generally means **lower CPU scheduling priority**.

7. renice

renice changes the niceness of an existing process.

```
renice 10 -p 1234
```

This changes the niceness of process 1234 to 10.

8. bg

bg resumes a stopped job and runs it in the **background**.

For example:

```
./program
```

Press:

```
Ctrl + Z
```

to stop/suspend it, then:

```
bg
```

to continue it in the background.

You can also specify a job:

```
bg %1
```

9. fg

fg brings a background job to the **foreground**.

```
fg
```

or:

```
fg %1
```

Quick example of job control

```
./program
```

Press **Ctrl + Z** → process is stopped.

```
bg
```

→ process continues in the background.

```
fg
```

→ process returns to the foreground.

In short

- top → monitor processes
- ps → list processes
- kill → send signals to processes
- wait → wait for a process/job
- sleep → pause execution
- nice → start with changed priority

- renice → change priority of an existing process
- bg → run stopped job in background
- fg → bring job to foreground

Exp3 :

ChatGPT said:

a) Commands for Sending Messages to Logged-in Users

Linux provides several commands to **identify logged-in users and communicate with them**.

Command	Purpose	Example
who	Shows users currently logged in.	who
cat	Displays or combines file contents; can be used with device/terminal files to send text.	cat file.txt
wall	Sends a message to **all logged-in users**.	wall "System will shut down soon"
write	Sends a message to a **specific logged-in user**.	write user1
mesg	Controls whether other users can write messages to your terminal.	mesg y

1. who

Displays information about users currently logged into the system.

```
who
```

Typical output:

```
student   pts/0   2026-09-02 09:30
admin     pts/1   2026-09-02 09:45
```

It shows the **username, terminal, login time**, and sometimes the remote host.

2. cat

cat is mainly used to **display and concatenate files**.

```
cat file.txt
```

It can also be used to send input to a terminal/device file when appropriate, although write and wall are the standard commands for user messaging.

3. wall

wall (**write all**) broadcasts a message to all users currently logged in.

```
wall "System shutdown in 10 minutes"
```

All users whose terminals permit messages will receive the notification.

4. write

write sends a message to a **particular logged-in user**.

```
write username
```

Then type the message:

```
Hello, can you check the server?
```

Press **Ctrl+D** to finish.

You can specify a particular terminal if the user has multiple sessions:

```
write username pts/1
```

5. mesg

mesg controls whether other users can send messages to your terminal using commands such as write.

Allow messages:

```
mesg y
```

Disable messages:

```
mesg n
```

Check the current setting:

```
mesg
```

---

b) List Processes Attached to a Shared Memory Segment: ipcs

ipcs displays information about **System V IPC (Inter-Process Communication)** resources, including shared memory segments.

To display shared memory segments:

```
ipcs -m
```

It provides information such as:

- Shared memory ID
- Owner
- Permissions
- Size
- Number of attached processes
- Creation and last-access information

For example:

```
------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch
0x123456   32768      user       600        4096       2
```

Here, nattch indicates the number of processes currently attached to that shared memory segment.
To display detailed information about a particular shared memory segment:

```
ipcs -m -i 32768
```

Quick revision

- who → list logged-in users
- cat → display/concatenate file contents
- wall → message **all** logged-in users
- write → message **one** logged-in user
- mesg → allow/block terminal messages
- ipcs -m → list **shared memory segments** and their attached-process count (nattch)

EXP4:

ChatGPT said:

Below are simple **C and Python programs** to calculate the **Average Waiting Time** and **Average Turnaround Time** using the **First Come First Serve (FCFS)** CPU scheduling algorithm.

Formula

For FCFS, processes execute in the order in which they are entered.

- **Waiting Time (WT)** = Completion Time − Burst Time
- **Turnaround Time (TAT)** = Completion Time
- Assuming all processes arrive at time 0.

C Program

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[100], wt[100], tat[100];
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
    printf("Enter burst time for each process:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting time of first process is 0
    wt[0] = 0;

    // Calculate waiting time
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Calculate turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    // Calculate averages
    for (i = 0; i < n; i++) {
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    avg_wt /= n;
    avg_tat /= n;

    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

Example

For:

```
Number of processes = 4
Burst times = 5 3 8 6
```

The calculation is:

```
Process    BT    WT    TAT
P1          5     0      5
P2          3     5      8
P3          8     8     16
P4          6    16     22
```

Therefore:

```
Average Waiting Time    = (0 + 5 + 8 + 16) / 4
                        = 7.25
```

```
Average Turnaround Time = (5 + 8 + 16 + 22) / 4
                        = 12.75
```

Python Program

```
n = int(input("Enter number of processes: "))

bt = []

print("Enter burst time for each process:")
for i in range(n):
    bt.append(int(input(f"P{i + 1}: ")))

wt = [0] * n
tat = [0] * n

# Calculate waiting time
for i in range(1, n):
    wt[i] = wt[i - 1] + bt[i - 1]

# Calculate turnaround time
for i in range(n):
    tat[i] = wt[i] + bt[i]

# Calculate averages
avg_wt = sum(wt) / n
avg_tat = sum(tat) / n

print("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time")

for i in range(n):
    print(f"P{i + 1}\t{bt[i]}\t\t{wt[i]}\t\t{tat[i]}")

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")
```

Note: This basic FCFS program assumes that **all processes arrive at time 0** and are executed in the order P1, P2, ..., Pn.

Exp5:

SJF CPU Scheduling Program

Shortest Job First (SJF) selects the process with the **smallest burst time** first. The following programs use **non-preemptive SJF** and assume that all processes arrive at time 0.

Formulas

- **Waiting Time (WT)** = Turnaround Time − Burst Time
- **Turnaround Time (TAT)** = Waiting Time + Burst Time
- **Average WT** = Sum of WT / Number of Processes
- **Average TAT** = Sum of TAT / Number of Processes

C Program

```
#include <stdio.h>

int main() {
    int n, i, j, temp;
    int bt[100], wt[100], tat[100], p[100];
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time for each process:\n");
    for (i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort processes according to burst time
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                // Swap burst time
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                // Swap process number
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    // First process has zero waiting time
    wt[0] = 0;

    // Calculate waiting time
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Calculate turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    // Calculate averages
    for (i = 0; i < n; i++) {
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    avg_wt /= n;
    avg_tat /= n;

    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\n",
               p[i], bt[i], wt[i], tat[i]);
    }
```

```
    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

Example

Suppose:

```
Number of processes = 4
Burst times:
P1 = 6
P2 = 8
P3 = 7
P4 = 3
```

SJF order will be:

```
P4 → P1 → P3 → P2
```

The result is:

```
Process    BT    WT    TAT
P4          3     0      3
P1          6     3      9
P3          7     9     16
P2          8    16     24
```

Therefore:

```
Average Waiting Time    = (0 + 3 + 9 + 16) / 4
                        = 7.00

Average Turnaround Time = (3 + 9 + 16 + 24) / 4
                        = 13.00
```

Python Program

```
n = int(input("Enter number of processes: "))

processes = []

print("Enter burst time for each process:")

for i in range(n):
    bt = int(input(f"P{i + 1}: "))
    processes.append([i + 1, bt])

# Sort according to burst time
processes.sort(key=lambda x: x[1])

wt = [0] * n
tat = [0] * n

# Calculate waiting time
for i in range(1, n):
    wt[i] = wt[i - 1] + processes[i - 1][1]

# Calculate turnaround time
```

```
for i in range(n):
    tat[i] = wt[i] + processes[i][1]

# Calculate averages
avg_wt = sum(wt) / n
avg_tat = sum(tat) / n

print("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time")

for i in range(n):
    print(f"P{processes[i][0]}\t{processes[i][1]}\t\t"
          f"{wt[i]}\t\t{tat[i]}")

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")
```

SJF in one line

SJF → Arrange processes in ascending order of burst time → calculate WT → calculate TAT → calculate their averages.

Exp6:

C and Python programs for **non-preemptive Priority CPU Scheduling**. Here, a **smaller priority number means higher priority**. All processes are assumed to arrive at time 0.

C Program

```
#include <stdio.h>

int main() {
    int n, i, j;
    int p[20], bt[20], priority[20];
    int wt[20], tat[20];
    int temp;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        p[i] = i + 1;

        printf("\nEnter Burst Time for P%d: ", p[i]);
        scanf("%d", &bt[i]);

        printf("Enter Priority for P%d: ", p[i]);
        scanf("%d", &priority[i]);
    }

    // Sort processes according to priority
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (priority[i] > priority[j]) {
                // Swap priority
                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;
```

```
                // Swap burst time
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                // Swap process number
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    // Waiting time of first process is 0
    wt[0] = 0;

    // Calculate waiting time
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Calculate turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];

        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nProcess\tBurst Time\tPriority\tWaiting Time\tTurnaround Time\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n",
               p[i], bt[i], priority[i], wt[i], tat[i]);
    }

    avg_wt = avg_wt / n;
    avg_tat = avg_tat / n;

    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

Python Program

```
n = int(input("Enter number of processes: "))

processes = []

for i in range(n):
    bt = int(input(f"Enter Burst Time for P{i+1}: "))
    priority = int(input(f"Enter Priority for P{i+1}: "))

    processes.append([i + 1, bt, priority])

# Sort according to priority
# Smaller number = higher priority
processes.sort(key=lambda x: x[2])

waiting_time = 0
total_wt = 0
```

```
total_tat = 0

print("\nProcess\tBurst Time\tPriority\tWaiting Time\tTurnaround Time")

for process in processes:
    pid = process[0]
    bt = process[1]
    priority = process[2]

    # Turnaround Time = Waiting Time + Burst Time
    tat = waiting_time + bt

    print(f"P{pid}\t{bt}\t\t{priority}\t\t{waiting_time}\t\t{tat}")

    total_wt += waiting_time
    total_tat += tat

    waiting_time += bt

avg_wt = total_wt / n
avg_tat = total_tat / n

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")
```

The calculations use:
Waiting Time (WT) = time a process waits before execution.
Turnaround Time (TAT) = Waiting Time + Burst Time
Average WT = Sum of all waiting times / Number of processes
Average TAT = Sum of all turnaround times / Number of processes
For example, if P1, P2, and P3 have priorities **2, 1, 3**, the execution order is **P2 → P1 → P3**, because priority 1 is treated as the highest priority.

Exp7:

ChatGPT said:

Here are simple **C and Python programs** for **Round Robin (RR) CPU Scheduling**. The program assumes all processes arrive at time 0.

C Program

```
#include <stdio.h>

int main() {
    int n, tq;
    int bt[20], rem_bt[20], wt[20], tat[20];
    int i, time = 0, done;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
```

```
        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    // Round Robin scheduling
    while (1) {
        done = 1;

        for (i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0;

                if (rem bt[i] > tq) {
                    time += tq;
                    rem_bt[i] -= tq;
                } else {
                    time += rem_bt[i];

                    // Turnaround Time = completion time
                    tat[i] = time;

                    // Waiting Time = TAT - Burst Time
                    wt[i] = tat[i] - bt[i];

                    rem_bt[i] = 0;
                }
            }
        }

        if (done == 1)
            break;
    }

    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);

        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    avg_wt /= n;
    avg_tat /= n;

    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

Python Program

```
n = int(input("Enter number of processes: "))

burst_time = []

for i in range(n):
    bt = int(input(f"Enter Burst Time for P{i + 1}: "))
```

```
    burst_time.append(bt)

time_quantum = int(input("Enter Time Quantum: "))

remaining_time = burst_time.copy()
waiting time = [0] * n
turnaround_time = [0] * n

time = 0

# Round Robin scheduling
while True:
    done = True

    for i in range(n):
        if remaining time[i] > 0:
            done = False

            if remaining_time[i] > time_quantum:
                time += time_quantum
                remaining_time[i] -= time_quantum
            else:
                time += remaining_time[i]

                # Completion time = Turnaround time
                turnaround_time[i] = time

                # Waiting Time = TAT - Burst Time
                waiting_time[i] = turnaround_time[i] - burst_time[i]

                remaining_time[i] = 0

    if done:
        break

print("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time")

for i in range(n):
    print(f"P{i + 1}\t{burst_time[i]}\t\t"
          f"{waiting_time[i]}\t\t{turnaround_time[i]}")

avg_wt = sum(waiting_time) / n
avg_tat = sum(turnaround_time) / n

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")
```

Formulas

- **Waiting Time (WT)** = Turnaround Time − Burst Time
- **Turnaround Time (TAT)** = Completion Time − Arrival Time
- Since all processes arrive at 0: **TAT = Completion Time**
- **Average WT** = Total WT / n
- **Average TAT** = Total TAT / n

Example: For burst times 5, 4, 2 and time quantum 2, the processes execute in the order P1 → P2 → P3 → P1 → P2 → P1.

Exp8:

Write a C/Python program to implement Banker's Algorithm.

C and Python programs to implement the Banker's Algorithm for deadlock avoidance.

C Program

```
#include <stdio.h>

int main() {
    int n, m;
    int allocation[10][10], max[10][10], need[10][10];
    int available[10], work[10], finish[10];
    int safeSequence[10];
    int i, j, count = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &m);

    // Input Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }

    // Input Maximum Matrix
    printf("\nEnter Maximum Matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    // Input Available Resources
    printf("\nEnter Available Resources:\n");
    for (j = 0; j < m; j++) {
        scanf("%d", &available[j]);
        work[j] = available[j];
    }

    // Calculate Need Matrix
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    // Initialize finish array
    for (i = 0; i < n; i++) {
        finish[i] = 0;
    }

    // Find Safe Sequence
    while (count < n) {
        int found = 0;

        for (i = 0; i < n; i++) {
            if (finish[i] == 0) {
                int possible = 1;
```

```
                for (j = 0; j < m; j++) {
                    if (need[i][j] > work[j]) {
                        possible = 0;
                        break;
                    }
                }

                if (possible) {
                    // Process can execute
                    for (j = 0; j < m; j++) {
                        work[j] += allocation[i][j];
                    }

                    safeSequence[count] = i;
                    count++;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if (found == 0) {
            printf("\nSystem is NOT in a safe state.\n");
            return 0;
        }
    }

    printf("\nSystem is in a SAFE state.\n");
    printf("Safe Sequence: ");

    for (i = 0; i < n; i++) {
        printf("P%d", safeSequence[i]);

        if (i != n - 1)
            printf(" -> ");
    }

    printf("\n");

    return 0;
}
```

Python Program

```
n = int(input("Enter number of processes: "))
m = int(input("Enter number of resource types: "))

# Input Allocation Matrix
print("\nEnter Allocation Matrix:")
allocation = []

for i in range(n):
    row = list(map(int, input(f"P{i}: ").split()))
    allocation.append(row)

# Input Maximum Matrix
print("\nEnter Maximum Matrix:")
maximum = []

for i in range(n):
    row = list(map(int, input(f"P{i}: ").split()))
    maximum.append(row)
```

```
# Input Available Resources
print("\nEnter Available Resources:")
available = list(map(int, input().split()))

# Calculate Need Matrix
need = []

for i in range(n):
    row = []
    for j in range(m):
        row.append(maximum[i][j] - allocation[i][j])
    need.append(row)

print("\nNeed Matrix:")
for row in need:
    print(row)

# Initialize
work = available.copy()
finish = [False] * n
safe_sequence = []

# Banker's Algorithm
while len(safe_sequence) < n:
    found = False

    for i in range(n):
        if not finish[i]:

            # Check if Need[i] <= Work
            possible = True

            for j in range(m):
                if need[i][j] > work[j]:
                    possible = False
                    break

            if possible:
                # Process can complete
                for j in range(m):
                    work[j] += allocation[i][j]

                safe_sequence.append(i)
                finish[i] = True
                found = True

    if not found:
        break

# Display result
if len(safe_sequence) == n:
    print("\nSystem is in a SAFE state.")
    print("Safe Sequence:", end=" ")

    for i, process in enumerate(safe_sequence):
        print(f"P{process}", end="")
        if i != len(safe_sequence) - 1:
            print(" -> ", end="")

    print()
else:
    print("\nSystem is NOT in a safe state.")
```

Key steps of Banker's Algorithm

1. Calculate the **Need Matrix**:
   Need = Maximum − Allocation
2. Set:
   Work = Available
3. Find a process whose:
   Need <= Work
4. Allow that process to finish and release its allocated resources:
   Work = Work + Allocation
5. Repeat until all processes finish.
6. If all processes can finish, the system is in a **safe state** and a **safe sequence** is produced. Otherwise, the system is **unsafe**.

Exp9:

Basic memory management commands - df, free, vmstat, /proc/meminfo, htop.

1. ***You said:***

   Write a C/Python program to calculate average waiting time and Turnaround Time of n processes with Priority CPU scheduling algorithm.

2. ***ChatGPT said:***

   Below are simple **C and Python programs** for **non-preemptive Priority CPU Scheduling**. Here, a **smaller priority number means higher priority**. All processes are assumed to arrive at time 0.

C Program

```
#include <stdio.h>

int main() {
    int n, i, j;
    int p[20], bt[20], priority[20];
    int wt[20], tat[20];
    int temp;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        p[i] = i + 1;

        printf("\nEnter Burst Time for P%d: ", p[i]);
        scanf("%d", &bt[i]);
```

```
        printf("Enter Priority for P%d: ", p[i]);
        scanf("%d", &priority[i]);
    }

    // Sort processes according to priority
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (priority[i] > priority[j]) {
                // Swap priority
                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;

                // Swap burst time
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                // Swap process number
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    // Waiting time of first process is 0
    wt[0] = 0;

    // Calculate waiting time
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Calculate turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];

        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nProcess\tBurst Time\tPriority\tWaiting Time\tTurnaround Time\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n",
               p[i], bt[i], priority[i], wt[i], tat[i]);
    }

    avg_wt = avg_wt / n;
    avg_tat = avg_tat / n;

    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

Python Program

```
n = int(input("Enter number of processes: "))

processes = []
```

```
for i in range(n):
    bt = int(input(f"Enter Burst Time for P{i+1}: "))
    priority = int(input(f"Enter Priority for P{i+1}: "))

    processes.append([i + 1, bt, priority])

# Sort according to priority
# Smaller number = higher priority
processes.sort(key=lambda x: x[2])

waiting_time = 0
total_wt = 0
total_tat = 0

print("\nProcess\tBurst Time\tPriority\tWaiting Time\tTurnaround Time")

for process in processes:
    pid = process[0]
    bt = process[1]
    priority = process[2]

    # Turnaround Time = Waiting Time + Burst Time
    tat = waiting_time + bt

    print(f"P{pid}\t{bt}\t\t{priority}\t\t{waiting_time}\t\t{tat}")

    total_wt += waiting_time
    total_tat += tat

    waiting_time += bt

avg_wt = total_wt / n
avg_tat = total_tat / n

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")
```

The calculations use:
Waiting Time (WT) = time a process waits before execution.
Turnaround Time (TAT) = Waiting Time + Burst Time
Average WT = Sum of all waiting times / Number of processes
Average TAT = Sum of all turnaround times / Number of processes
For example, if P1, P2, and P3 have priorities **2, 1, 3**, the execution order is **P2 → P1 → P3**, because priority 1 is treated as the highest priority.

3. ***You said:***

Write a C/Python program to calculate average waiting time and Turnaround Time of n processes with Round Robin (RR) CPU scheduling algorithm.

4. ***ChatGPT said:***

Here are simple **C and Python programs** for **Round Robin (RR) CPU Scheduling**. The program assumes all processes arrive at time 0.

C Program

```
#include <stdio.h>

int main() {
    int n, tq;
    int bt[20], rem_bt[20], wt[20], tat[20];
    int i, time = 0, done;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    // Round Robin scheduling
    while (1) {
        done = 1;

        for (i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0;

                if (rem_bt[i] > tq) {
                    time += tq;
                    rem_bt[i] -= tq;
                } else {
                    time += rem_bt[i];

                    // Turnaround Time = completion time
                    tat[i] = time;

                    // Waiting Time = TAT - Burst Time
                    wt[i] = tat[i] - bt[i];

                    rem_bt[i] = 0;
                }
            }
        }

        if (done == 1)
            break;
    }

    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\n",
               i + 1, bt[i], wt[i], tat[i]);

        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    avg_wt /= n;
    avg_tat /= n;
```

```
    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat);

    return 0;
}
```

Python Program

```
n = int(input("Enter number of processes: "))

burst_time = []

for i in range(n):
    bt = int(input(f"Enter Burst Time for P{i + 1}: "))
    burst_time.append(bt)

time_quantum = int(input("Enter Time Quantum: "))

remaining_time = burst_time.copy()
waiting_time = [0] * n
turnaround_time = [0] * n

time = 0

# Round Robin scheduling
while True:
    done = True

    for i in range(n):
        if remaining_time[i] > 0:
            done = False

            if remaining_time[i] > time_quantum:
                time += time_quantum
                remaining_time[i] -= time_quantum
            else:
                time += remaining_time[i]

                # Completion time = Turnaround time
                turnaround_time[i] = time

                # Waiting Time = TAT - Burst Time
                waiting_time[i] = turnaround_time[i] - burst_time[i]

                remaining_time[i] = 0

    if done:
        break

print("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time")

for i in range(n):
    print(f"P{i + 1}\t{burst_time[i]}\t\t"
          f"{waiting_time[i]}\t\t{turnaround_time[i]}")

avg_wt = sum(waiting_time) / n
avg_tat = sum(turnaround_time) / n

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")
```

Formulas

- **Waiting Time (WT)** = Turnaround Time − Burst Time
- **Turnaround Time (TAT)** = Completion Time − Arrival Time
- Since all processes arrive at 0: **TAT = Completion Time**
- **Average WT** = Total WT / n
- **Average TAT** = Total TAT / n

Example: For burst times 5, 4, 2 and time quantum 2, the processes execute in the order P1 → P2 → P3 → P1 → P2 → P1.

5. You said:

Write a C/Python program to implement Banker's Algorithm.

6. ChatGPT said:

Below are simple **C and Python programs to implement the Banker's Algorithm** for deadlock avoidance.

C Program

```
#include <stdio.h>

int main() {
    int n, m;
    int allocation[10][10], max[10][10], need[10][10];
    int available[10], work[10], finish[10];
    int safeSequence[10];
    int i, j, count = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &m);

    // Input Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }

    // Input Maximum Matrix
    printf("\nEnter Maximum Matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    // Input Available Resources
    printf("\nEnter Available Resources:\n");
    for (j = 0; j < m; j++) {
        scanf("%d", &available[j]);
```

```
        work[j] = available[j];
    }

    // Calculate Need Matrix
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    // Initialize finish array
    for (i = 0; i < n; i++) {
        finish[i] = 0;
    }

    // Find Safe Sequence
    while (count < n) {
        int found = 0;

        for (i = 0; i < n; i++) {
            if (finish[i] == 0) {
                int possible = 1;

                for (j = 0; j < m; j++) {
                    if (need[i][j] > work[j]) {
                        possible = 0;
                        break;
                    }
                }

                if (possible) {
                    // Process can execute
                    for (j = 0; j < m; j++) {
                        work[j] += allocation[i][j];
                    }

                    safeSequence[count] = i;
                    count++;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if (found == 0) {
            printf("\nSystem is NOT in a safe state.\n");
            return 0;
        }
    }

    printf("\nSystem is in a SAFE state.\n");
    printf("Safe Sequence: ");

    for (i = 0; i < n; i++) {
        printf("P%d", safeSequence[i]);

        if (i != n - 1)
            printf(" -> ");
    }

    printf("\n");

    return 0;
```

```
}
```

Python Program

```
n = int(input("Enter number of processes: "))
m = int(input("Enter number of resource types: "))

# Input Allocation Matrix
print("\nEnter Allocation Matrix:")
allocation = []

for i in range(n):
    row = list(map(int, input(f"P{i}: ").split()))
    allocation.append(row)

# Input Maximum Matrix
print("\nEnter Maximum Matrix:")
maximum = []

for i in range(n):
    row = list(map(int, input(f"P{i}: ").split()))
    maximum.append(row)

# Input Available Resources
print("\nEnter Available Resources:")
available = list(map(int, input().split()))

# Calculate Need Matrix
need = []

for i in range(n):
    row = []
    for j in range(m):
        row.append(maximum[i][j] - allocation[i][j])
    need.append(row)

print("\nNeed Matrix:")
for row in need:
    print(row)

# Initialize
work = available.copy()
finish = [False] * n
safe_sequence = []

# Banker's Algorithm
while len(safe_sequence) < n:
    found = False

    for i in range(n):
        if not finish[i]:

            # Check if Need[i] <= Work
            possible = True

            for j in range(m):
                if need[i][j] > work[j]:
                    possible = False
                    break

            if possible:
                # Process can complete
                for j in range(m):
```

```
                    work[j] += allocation[i][j]

                safe_sequence.append(i)
                finish[i] = True
                found = True

    if not found:
        break

# Display result
if len(safe_sequence) == n:
    print("\nSystem is in a SAFE state.")
    print("Safe Sequence:", end=" ")

    for i, process in enumerate(safe_sequence):
        print(f"P{process}", end="")
        if i != len(safe_sequence) - 1:
            print(" -> ", end="")

    print()
else:
    print("\nSystem is NOT in a safe state.")
```

Key steps of Banker's Algorithm

1. Calculate the **Need Matrix**:

Need = Maximum − Allocation

2. Set:

Work = Available

3. Find a process whose:

Need <= Work

4. Allow that process to finish and release its allocated resources:

Work = Work + Allocation

5. Repeat until all processes finish.
6. If all processes can finish, the system is in a **safe state** and a **safe sequence** is produced. Otherwise, the system is **unsafe**.

You said:

Basic memory management commands - df, free, vmstat, /proc/meminfo, htop.

ChatGPT said:

Basic Memory Management Commands in Linux

These commands are commonly used to **monitor memory usage, disk space, swap, and system performance**.

1. df

df stands for **Disk Free**. It displays the amount of free and used **disk space** on mounted file systems.

```
df
```

For a more readable output:

```
df -h
```

Example:

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   25G   23G  53% /
```

Important options:

- df -h — Human-readable format
- df -T — Shows file system type
- df -i — Shows inode usage

Note: df monitors **disk/storage**, not RAM.

---

2. free

free displays information about **RAM and swap memory**.

```
free
```

For human-readable output:

```
free -h
```

Example:

```
               total   used   free   shared  buff/cache  available
Mem:            7.7G   3.2G   1.5G     200M       3.0G       4.1G
Swap:           2.0G   100M   1.9G
```

Important options:

- free -h — Human-readable output
- free -m — Display values in MB
- free -g — Display values in GB
- free -s 2 — Refresh every 2 seconds

---

3. vmstat

vmstat stands for **Virtual Memory Statistics**. It provides information about processes, memory, paging, block I/O, and CPU activity.

```
vmstat
```

To continuously monitor the system:

```
vmstat 2
```

This updates every **2 seconds**.
Example:

```
procs   memory        swap   io       system      cpu
 r  b   swpd  free    si  so  bi  bo   in   cs   us sy id
 1  0      0  1500     0   0   5  10  100  200    5  2 93
```

Important fields:

- free — Free memory
- si — Memory swapped in
- so — Memory swapped out
- us — CPU time spent on user processes
- sy — CPU time spent on system processes
- id — CPU idle time

---

4. /proc/meminfo
/proc/meminfo is a **virtual file** containing detailed information about the system's memory.
View it using:

```
cat /proc/meminfo
```

For example:

```
MemTotal:        8023456 kB
MemFree:         1536000 kB
MemAvailable:    4200000 kB
Buffers:          120000 kB
Cached:          2500000 kB
SwapTotal:       2097148 kB
SwapFree:        1990000 kB
```

Some important entries:

- MemTotal — Total physical RAM
- MemFree — Currently unused RAM
- MemAvailable — RAM available for new applications
- Buffers — Memory used for kernel buffers
- Cached — Memory used for file-system cache
- SwapTotal — Total swap space
- SwapFree — Available swap space

You can search for a specific value:

```
grep MemTotal /proc/meminfo
```

---

5. htop

htop is an **interactive process and system monitor**. It displays CPU usage, memory usage, swap usage, and running processes.

Run:

```
htop
```

If it is not installed:

```
sudo apt install htop
```

The top portion generally shows:

- CPU utilization
- RAM utilization
- Swap utilization

The process list shows:

- Process ID (PID)
- User
- CPU usage
- Memory usage
- Process name

Useful keys inside htop:

- F2 — Setup
- F3 — Search
- F4 — Filter
- F5 — Tree view
- F6 — Sort
- F9 — Kill a process
- F10 — Quit

Quick Comparison

Command	Purpose
df -h	Check **disk space**
free -h	Check **RAM and swap**

Command	Purpose
vmstat	Monitor **memory, processes, I/O and CPU**
cat /proc/meminfo	View **detailed memory information**
htop	Interactive **process and resource monitoring**

Common commands to remember

```
df -h
free -h
vmstat 2
cat /proc/meminfo
htop
```

These are useful basic Linux commands for **memory/resource management and system monitoring**.

Exp10 :
Write a C/Python program on First In First Out (FIFO) Page Replacement algorithm.

FIFO Page Replacement Algorithm — Python

```
# FIFO Page Replacement Algorithm

def fifo_page_replacement(pages, frames):
    memory = []
    page_faults = 0
    page_hits = 0
    pointer = 0

    print("\nPage\tFrames\t\tStatus")
    print("-" * 40)

    for page in pages:
        if page in memory:
            page_hits += 1
            status = "Hit"
        else:
            page_faults += 1
            status = "Fault"

            if len(memory) < frames:
                memory.append(page)
            else:
                memory[pointer] = page
                pointer = (pointer + 1) % frames

        print(f"{page}\t{memory}\t\t{status}")

    print("\nTotal Page Faults :", page_faults)
    print("Total Page Hits   :", page_hits)


# Input
```

```
pages = list(map(int, input("Enter the page reference string: ").split()))
frames = int(input("Enter the number of frames: "))

# Function call
fifo_page_replacement(pages, frames)
```

Example Input

```
Enter the page reference string: 1 2 3 4 1 2 5 1 2 3 4 5
Enter the number of frames: 3
```

Example Output

```
Page    Frames          Status
---------------------------------------
1       [1]             Fault
2       [1, 2]          Fault
3       [1, 2, 3]       Fault
4       [4, 2, 3]       Fault
1       [4, 1, 3]       Fault
2       [4, 1, 2]       Fault
5       [5, 1, 2]       Fault
1       [5, 1, 2]       Hit
2       [5, 1, 2]       Hit
3       [5, 3, 2]       Fault
4       [5, 3, 4]       Fault
5       [5, 3, 4]       Hit

Total Page Faults : 9
Total Page Hits   : 3
```

FIFO principle: The page that entered memory **first** is replaced first when a page fault occurs and all frames are occupied.

FIFO Page Replacement Algorithm — C

```c
#include <stdio.h>

int main()
{
    int pages[50], frames[10];
    int n, f;
    int i, j, k;
    int pageFaults = 0, pageHits = 0;
    int pointer = 0, found;

    printf("Enter the number of pages: ");
    scanf("%d", &n);

    printf("Enter the page reference string:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter the number of frames: ");
    scanf("%d", &f);

    for (i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nPage\tFrames\t\tStatus\n");
    printf("---------------------------------------\n");
```

```
    for (i = 0; i < n; i++)
    {
        found = 0;

        // Check whether page is already present
        for (j = 0; j < f; j++)
        {
            if (frames[j] == pages[i])
            {
                found = 1;
                break;
            }
        }

        if (found)
        {
            pageHits++;
            printf("%d\t", pages[i]);

            for (k = 0; k < f; k++)
            {
                if (frames[k] != -1)
                    printf("%d ", frames[k]);
            }

            printf("\t\tHit\n");
        }
        else
        {
            pageFaults++;

            // Replace the oldest page
            frames[pointer] = pages[i];
            pointer = (pointer + 1) % f;

            printf("%d\t", pages[i]);

            for (k = 0; k < f; k++)
            {
                if (frames[k] != -1)
                    printf("%d ", frames[k]);
            }

            printf("\t\tFault\n");
        }
    }

    printf("\nTotal Page Faults = %d\n", pageFaults);
    printf("Total Page Hits   = %d\n", pageHits);

    return 0;
}
```

Sample Input

```
Enter the number of pages: 12
Enter the page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter the number of frames: 3
```

Sample Output

```
Page    Frames          Status
---------------------------------------
1       1               Fault
2       1 2             Fault
3       1 2 3           Fault
4       4 2 3           Fault
1       4 1 3           Fault
2       4 1 2           Fault
5       5 1 2           Fault
1       5 1 2           Hit
2       5 1 2           Hit
3       5 3 2           Fault
4       5 3 4           Fault
5       5 3 4           Hit

Total Page Faults = 9
Total Page Hits   = 3
```

Algorithm

1. Read the page reference string and number of frames.
2. Initially, all frames are empty.
3. For each page, check whether it is already present in a frame.
4. If present, it is a **page hit**.
5. If not present, it is a **page fault**.
6. If a frame is available, insert the page.
7. If all frames are occupied, replace the page that entered memory first.
8. Move the FIFO pointer to the next frame.
9. Continue until all pages are processed.
10. Display the total number of page faults and page hits.

Time complexity: $O(n \times f)$, where n is the number of page references and f is the number of frames.